Stellart Games

First Defense

ATRIA Project

Matteo Dupond
Guillaume Dufour
Lilian Labat
Nelson Rudent
Luan Ray-Chagas

June 16, 2024

Contents

I : Introduction **3**

II : Body **4**

1 Matteo Dupond 4
1.1 Website 4
1.2 IA 8
2 Guillaume Dufour 13
2.1 User Interface 13
2.2 Assets 20
3 Nelson Rudent 22
3.1 Gameplay 22
3.2 Multiplayer 24
3.3 Game Design 25
4 Lilian Labat 29
4.1 Procedural Generation 29
4.2 Multiplayer 30
5 Luan Ray-Chagas 34
5.1 Sound 34
5.2 Networking 36

III : Conclusion **37**

IV : Annexe **38**

Introduction

This comprehensive report details the technical development progress of our ambitious project, Atria. It serves as a testament to the collective efforts and dedication of our team members, each contributing their unique expertise to shape the evolution of this futuristic space first-person shooter (FPS) game.

In the following pages, you will find an in-depth analysis of our project's progressive development, carefully organized by team members with their respective branches. From gameplay mechanics to storytelling elements, multiplayer integration to user interface design, every facet of Atria's development process is diligently documented to provide an accurate view of our progress.

As you navigate through the report, you will gain insight into the innovative solutions, technical challenges, and creative decisions that have shaped the evolution of Atria. Each section highlights the contributions of individual team members, showcasing their expertise and dedication to advancing our shared vision.

We invite you to look into the complications of Atria's development progress, exploring the collaborative efforts and innovative strategies that have propelled our project forward. From conceptualization to implementation, our team remains committed to delivering an immersive and unforgettable gaming experience. We provide you with a table of contents that you can follow to quickly find your way throughout the group member's work and the branches they participate in.

II : Body

1 Matteo Dupond

1.1 Website

Creating a captivating online presence for Atria was a task I undertook with meticulous attention to detail and a commitment to delivering an immersive experience for visitors. From the moment users land on the website, which is the first interaction they get with our game, I wanted to ensure they were greeted with a sense of anticipation and excitement, which is why I incorporated a visually engaging loading animation. This animation not only serves a practical purpose by indicating that content is loading but also sets the tone for the futuristic and dynamic universe of Atria.

As users explore the website, their attention is drawn to a prominent download button strategically positioned as the gateway to experiencing Atria. This strategic placement aims to capture visitors' attention and entice them to take the next step in their journey with the game. The download process is meticulously designed to be intuitive and user-friendly, accommodating individuals of varying technical proficiencies. From novice players to seasoned gamers, the seamless transition from browsing to gameplay ensures that every user can effortlessly access and enjoy the immersive world of Atria.

Once users click on the download button, they are guided through a streamlined process that minimizes complexity and maximizes convenience. Clear instructions and visual cues accompany users every step of the way, simplifying the download process and eliminating potential barriers to entry. This intuitive approach not only enhances user experience but also fosters a sense of accessibility, welcoming players of all backgrounds to embark on their adventure in Atria.

The seamless integration of the download mechanism into the website's interface reflects our commitment to providing an exceptional user experience. By prioritizing simplicity and ease of use, we ensure that every visitor can effortlessly transition from exploration to engagement, setting the stage for an immersive and enjoyable gaming experience in the world of Atria.

In addition to facilitating access to the game, the website serves as a platform to showcase the talented individuals who contributed to the creation of Atria. To achieve this, I implemented a carousel feature that prominently displays each team member. This carousel not only highlights the diverse skills and expertise of the team but also

emphasizes the collaborative effort that went into bringing the game to life. By providing insight into the team behind the game, we aim to foster a sense of connection and trust with our audience.

Recognizing the importance of transparency and personal connection, I dedicated separate pages to each team member. These individual pages offer visitors a deeper understanding of the role and contributions of each team member, allowing them to appreciate the unique talents that have shaped Atria. From developers to designers, each team member's page provides valuable insights into their background, expertise, and passion for gaming.

Furthermore, the website serves as a hub for engaging with the Atria community and staying updated on the latest news and developments. Whether it's through blog posts, social media integration, or interactive forums, we strive to create an inclusive and interactive space where players can connect with each other and with the team behind the game.

The website I developed for Atria goes beyond a mere promotional tool. It is a dynamic and immersive platform that invites visitors to explore the world of the game, connect with its creators, and become part of a vibrant gaming community. With a focus on user experience, aesthetics, and functionality, I aimed to create a website that reflects the quality and excitement of the game itself.

In developing the website, I also placed significant emphasis on ensuring it was optimized for various devices and screen sizes. Given the diverse range of devices that users might employ to access the site, from desktop computers to smartphones, a responsive design was paramount. This ensures that the website's functionality and visual appeal remain consistent and effective, regardless of the device. The responsive design includes fluid grid layouts, flexible images, and CSS media queries, all working together to adapt the website's content to different screen sizes and orientations seamlessly.

One of the standout features of the website is its dynamic and interactive map of Atria's game world. This map allows users to explore different regions of the game environment directly from the website, providing an engaging preview of the in-game experience. Users can click on various locations to learn more about key landmarks, missions, and characters, fostering a deeper connection to the game before even downloading it. This interactive element not only enhances user engagement but also serves as an effective marketing tool by showcasing the game's depth and complexity.

To further enrich the user experience, I integrated multimedia content throughout the website. High-resolution images, video trailers, and gameplay footage are seamlessly embedded, providing visitors with a rich, immersive preview of what Atria has to offer. These multimedia elements are carefully curated to highlight the game's stunning graphics, intricate storyline, and innovative gameplay mechanics. By offering a multimedia-rich experience, the website not only captures the essence of Atria but also appeals to a broader audience, from casual visitors to hardcore gaming enthusiasts.

Another critical aspect of the website is its robust support and feedback system. Recognizing the importance of community and player support, I implemented a comprehensive support section featuring FAQs, troubleshooting guides, and a ticketing system for personalized assistance. Additionally, a feedback form allows users to share their thoughts and suggestions directly with the development team. This two-way communication channel ensures that the team can address user concerns promptly and gather valuable insights for future updates and improvements.

SEO (Search Engine Optimization) was also a key consideration in the website's development. By employing best practices in SEO, such as optimizing page load speeds, using descriptive meta tags, and creating keyword-rich content, I ensured that the website would rank highly in search engine results. This not only increases the website's visibility but also drives more organic traffic, ultimately leading to higher download rates and a larger player base for Atria.

The integration of social media features further enhances the website's connectivity and outreach. Social media buttons and widgets enable users to share content effortlessly and follow Atria's latest updates on platforms like Facebook, Twitter, and Instagram. This social media integration helps build a sense of community and keeps players engaged with ongoing game developments and events. Additionally, embedding live social media feeds on the website ensures that users can stay updated with the latest news and interact with the game's community in real time.

To ensure the website remains relevant and engaging, I also established a content management system (CMS) that allows the team to update content easily and regularly. This CMS enables the team to post news articles, updates, and new media content without requiring extensive technical knowledge. By keeping the website content fresh and current, we maintain user interest and encourage repeat visits,

fostering a loyal community around Atria.

Moreover, the website incorporates advanced analytics tools to track user behavior and engagement. By analyzing data such as page views, click-through rates, and user feedback, we can continually refine and optimize the website to better meet the needs and preferences of our audience. These insights help us understand which aspects of the site are most effective in driving downloads and engagement, allowing us to make data-driven decisions to enhance the overall user experience.

The website also features an integrated blog where the development team regularly posts updates, behind-the-scenes insights, and news about upcoming features and events. This blog not only keeps the community informed but also fosters a deeper connection between the players and the creators. By sharing stories from the development process, we can build anticipation for future updates and maintain a transparent dialogue with our audience.

In addition to the blog, the website hosts a forum where players can discuss strategies, share tips, and form guilds or alliances. This forum is moderated to ensure a positive and respectful community atmosphere. By providing a dedicated space for player interaction, we enhance the sense of community and encourage collaborative gameplay experiences.

To attract and retain a global audience, the website supports multiple languages. This localization effort ensures that non-English speaking users can navigate the site and access content in their preferred language, making Atria more accessible to players worldwide. Language options are easily selectable from a dropdown menu, allowing users to switch seamlessly between different translations.

For promotional campaigns, the website includes dedicated landing pages for special events, updates, and seasonal promotions. These landing pages are designed to capture the attention of visitors with visually appealing graphics and compelling calls to action. By creating targeted content for specific events, we can drive engagement and participation in limited-time offers and in-game events.

Furthermore, the website is equipped with an email newsletter subscription option. By subscribing, users can receive regular updates directly to their inbox, including news about game updates, special events, and exclusive offers. This email marketing strategy helps us maintain a direct line of communication with our players and keep them engaged with the latest developments in Atria.

To ensure the website remains secure and reliable, we implemented robust security measures such as SSL encryption, regular security audits, and server redundancy. These measures protect user data and ensure that the site remains accessible even during high traffic periods. By prioritizing security and reliability, we build trust with our audience and provide a safe browsing experience.

Incorporating accessibility features was also a priority in the website's design. We ensured that the site complies with web accessibility standards, making it usable for individuals with disabilities. Features such as alt text for images, keyboard navigation, and screen reader compatibility ensure that everyone, regardless of their abilities, can access and enjoy the content on the website.

Finally, the website includes a comprehensive media kit for press and influencers. This kit provides high-resolution images, logos, press releases, and other assets that can be used to create content about Atria. By providing these resources, we facilitate media coverage and influencer partnerships, further expanding the game's reach and visibility.

Overall, the website for Atria is a multifaceted platform designed to engage, inform, and support its users. Through a combination of responsive design, interactive elements, multimedia content, robust support systems, and SEO strategies, the website effectively captures the spirit of Atria and invites users to immerse themselves in its world. By prioritizing user experience and leveraging advanced web technologies, the website stands as a testament to the quality and innovation that Atria represents.

1.2 IA

In addition to crafting an engaging website for Atria, I also took interest in promoting a feature of the project that could pique the interest of visitors. Since Artificial Intelligence is in its development explosion era, I decided to highlight it on the front-end. This could multiply the number of visitors and increase the overall interest, potentially pushing them to download the game and try it out for themselves. Hence, I delved into the realm of game development by implementing a pathfinding algorithm using the renowned A* algorithm. This algorithm serves a crucial role in enhancing the gameplay experience by enabling intelligent and efficient navigation for characters within the game world, which is what I want to promote on the website.

The A* algorithm, known for its effectiveness in finding the shortest path between two points, was the natural choice for Atria's pathfinding needs. By leveraging this algorithm, I aimed to create an environment where characters can navigate dynamically and adaptively, responding to changing conditions and obstacles in real-time.

Implementing the A* algorithm involved several key steps. First, I needed to represent the game world as a graph, with nodes representing key points of interest and edges representing possible paths between them. This required careful consideration of the game's terrain, obstacles, and other environmental factors that could affect movement.

Next, I implemented the core logic of the A* algorithm, which involves iteratively evaluating possible paths from a starting point to a target point. At each iteration, the algorithm selects the most promising path based on a heuristic function that estimates the cost of reaching the target from the current node. This process continues until the algorithm finds the shortest path or exhausts all possible options.

To ensure smooth and efficient pathfinding in real-time, I optimized the algorithm's performance by implementing techniques such as caching, pruning, and heuristic adjustments. These optimizations helped reduce computational overhead and improve responsiveness, ensuring a seamless gameplay experience for players.

Once the pathfinding algorithm was integrated into the game engine, characters within Atria gained the ability to navigate complex environments with ease. Whether exploring procedurally generated maps, engaging in combat, or completing objectives, characters can now move intelligently and autonomously, enhancing the overall immersion and depth of the gameplay experience.

Beyond the fundamental implementation of the A* algorithm, I explored advanced techniques to further enhance the AI's capabilities. One such technique was dynamic obstacle avoidance, which allows characters to react to moving objects and other players in real-time. By incorporating a layer of predictive analytics, the AI could anticipate potential collisions and adjust paths accordingly, providing a more fluid and natural movement pattern for in-game characters. This level of sophistication not only improves gameplay but also adds a layer of realism that players appreciate.

Moreover, I integrated environmental awareness into the AI's decision-making process. Characters were programmed to recognize and utilize various elements of the game environment to their advantage.

For example, AI-controlled characters could take cover during combat, navigate around hazardous areas, and exploit terrain features to gain strategic advantages. This level of environmental interaction makes the AI appear more intelligent and lifelike, greatly enhancing the player's immersion in the game.

To further refine the AI's performance, I implemented machine learning techniques to allow the pathfinding algorithm to adapt and improve over time. By analyzing player behavior and common movement patterns, the AI could adjust its strategies to provide more challenging and unpredictable gameplay. This adaptive learning capability ensures that the game remains engaging and challenging for players of all skill levels.

In addition to improving the gameplay experience, the AI's capabilities were showcased on the website through interactive demos and visualizations. Visitors could see firsthand how the AI navigates complex environments, responds to dynamic obstacles, and learns from player interactions. These demos serve not only as a promotional tool but also as an educational resource, offering insights into the sophisticated technology behind Atria.

To support the ongoing development and optimization of the AI, I also established a comprehensive testing framework. This framework includes automated testing routines that simulate various in-game scenarios, ensuring the AI performs reliably under different conditions. Regular performance reviews and updates based on player feedback are integral to maintaining the AI's effectiveness and keeping the game experience fresh and engaging.

Another important aspect of promoting AI on the website is transparency. By providing detailed explanations of the AI algorithms and techniques used, I aimed to build trust and credibility with our audience. Blog posts, technical articles, and video tutorials delve into the intricacies of the A* algorithm and its implementation in Atria. This transparency not only demystifies the technology for curious players but also positions Atria as a cutting-edge game that leverages advanced AI to enhance gameplay.

The AI's capabilities also extend to NPC (non-player character) interactions, adding depth to the game's narrative and social dynamics. NPCs in Atria are designed to exhibit realistic behaviors, engage in meaningful conversations, and react appropriately to the player's actions. This level of interaction enriches the storytelling aspect of the game, making the world of Atria feel alive and responsive. Promot-

ing these AI-driven interactions on the website highlights the game's complexity and the immersive experience it offers.

In the realm of combat, the AI's advanced pathfinding and decision-making capabilities shine. Characters can strategize, flank opponents, and utilize cover effectively, providing a challenging and realistic combat experience. The AI can also adapt to the player's tactics, ensuring that each encounter feels unique and requires thoughtful strategy. This dynamic combat system, powered by advanced AI, significantly enhances the excitement and replayability of Atria.

Additionally, the AI plays a crucial role in managing the game's economy and resource allocation. Characters can make intelligent decisions about gathering resources, crafting items, and managing inventories, contributing to a balanced and engaging gameplay experience. By showcasing these economic AI features on the website, potential players can appreciate the depth and sophistication of Atria's game mechanics.

To ensure the AI remains at the forefront of game development, I also focused on continuous learning and improvement. The AI is designed to receive updates and enhancements based on player feedback and new advancements in AI research. This commitment to ongoing development ensures that Atria remains a cutting-edge game that evolves with the latest technological trends.

Furthermore, the AI's role extends to the game's social interactions and community dynamics. Characters in Atria can form alliances, engage in diplomacy, and participate in complex social networks. This social AI adds another layer of depth to the game, creating a living world where player decisions have far-reaching consequences. Highlighting these social AI features on the website underscores Atria's unique position in the gaming landscape.

To facilitate a deeper understanding of the AI's capabilities, I also implemented an AI showcase section on the website. This section features detailed breakdowns of various AI components, interactive demos, and behind-the-scenes insights from the development team. By offering this level of detail, we aim to engage both casual gamers and tech enthusiasts, providing a comprehensive view of the innovative AI driving Atria.

Moreover, I integrated AI-driven analytics into the website to enhance user engagement and retention. By analyzing visitor behavior and preferences, the website can offer personalized content recommendations, targeted marketing campaigns, and optimized user ex-

periences. This AI-powered personalization ensures that each visitor feels valued and engaged, increasing the likelihood of conversion and long-term retention.

To further capitalize on the AI's marketing potential, I also developed AI-generated content for the website. Using natural language processing (NLP) algorithms, the website can automatically generate blog posts, news updates, and social media content that resonates with our audience. This AI-driven content creation not only saves time but also ensures a consistent and engaging brand voice across all communication channels.

In the realm of customer support, AI-powered chatbots and virtual assistants provide instant assistance to website visitors. These AI tools can handle a wide range of queries, from technical support to gameplay tips, ensuring that users receive timely and accurate information. By integrating these AI-driven support features, we enhance the overall user experience and build a strong foundation for community engagement.

The AI's influence extends to the website's security and data protection measures. Advanced machine learning algorithms monitor website traffic and user activity for potential security threats, ensuring a safe and secure browsing experience. By promoting these AI-driven security features, we reassure visitors that their data and privacy are protected.

Finally, the AI's capabilities in Atria are not static but continually evolving. Through regular updates and enhancements, the AI adapts to new gameplay challenges and player behaviors, ensuring that Atria remains a dynamic and engaging experience. This commitment to AI innovation positions Atria as a leader in the gaming industry, attracting a dedicated and enthusiastic player base.

In summary, the integration of advanced AI, particularly the A* pathfinding algorithm, significantly enhances the gameplay and overall user experience in Atria. By promoting these AI features on the website, we not only attract tech-savvy visitors but also demonstrate the innovative spirit behind the game's development. This strategic focus on AI serves as a powerful marketing tool, drawing interest and driving downloads, while also setting Atria apart as a game that offers both cutting-edge technology and an immersive, dynamic gaming environment.

2 Guillaume Dufour

2.1 User Interface

Once the game has been downloaded from the website created by Matteo, the Atria user interface will be the first thing players encounter when they try the game. Therefore, it was necessary to create a practical, functional, attractive, and intuitive interface to best guide new players and allow them to access the functions of their choice.

Upon entering the game, the player is presented with the options of a Play button, an Options button, a Credits button, and a Quit button.

If the player clicks on the Credits button, they are taken to a page listing the project collaborators (Lilian Labat, Matteo Dupond, Nelson Rudent, Luan Ray-Chagas, and Guillaume Dufour).

In the options menu, the user can choose to adjust the game volume with an On/Off switch, as well as the level of the volume, by clicking on the Sound button. They can also adjust the resolution as well as the display modes if they click on the Graphics button. Players also have the option to remap their keyboard binds in the Controls submenu. Finally, the Matchmaker button gets the player to a menu where they can refer a custom IP address, to make private games for example.

To implement sound management in my game, I started by adding a feature to mute and unmute the sound via a button. In the ‘$_{o}n_{c}heck_{b}utton_{p}ressed$‘$me$
$AudioStreamPlayer > (”MenuSwitch”).Play()$‘$.Next, Iuse$‘$HSlider$‘$and$‘$CheckB$

To handle volume changes, I created the ‘$_{o}n_{hs}lider_{v}alue_{c}hanged$‘$method, whichad$
$AudioStreamPlayer > (”MenuSwitch”).Play()$‘$toindicatethatthechangehasbeenr$

In the graphics menu, players can choose between different resolutions. The available resolutions are either 1280 by 720 pixels, or 1920 by 1080 pixels. The different display modes are Exclusive Fullscreen, where the game is maximized on screen and no other app can display over it, Fullscreen (also called Borderless Fullscreen) to get the game maximized but keep other apps displayed if necessary, and Windowed, which gets the game in windowed mode, to be able to put the game on another monitor, for example.

To implement window mode and resolution selection in Atria, I used ‘OptionButton‘ nodes for both the resolution and window mode settings. These nodes are exported for easy configuration in the editor, allowing the user to select their preferred settings from a dropdown menu. The available resolution options are stored in an array called ‘Resolutions‘, which includes values such as ”1920x1080” and

"1280x720". An integer variable, ‘resolutionIndex‘, keeps track of the selected resolution. The ‘$_on_option_button_item_selected$‘$method updatesthisindexwhene$

For managing window modes, I created an array called ‘Modes‘, which stores integer values representing different window modes such as windowed, fullscreen, and borderless. An integer variable, ‘index‘, keeps track of the currently selected window mode. The ‘$_on_window_mode_item_selected$‘

When the user confirms their choices by pressing the OK button, the ‘$_on_okpressed_graphics$‘$methodistriggered.Thismethodfirstsetsthewindowmodet$ $(Window.ModeEnum)Modes[index]$‘$.Itthenretrievestheselectedresolutionfromth$

To ensure the user interface reflects the current settings accurately, I implemented the ‘SetResolutionIndicator‘ method. This method retrieves the current window size with ‘DisplayServer.WindowGetSize()‘ and converts it into a string format. It then compares this string to the available resolutions in the ‘Resolutions‘ array. If a match is found, the ‘resolutionIndex‘ is updated to the corresponding index, ensuring the ‘OptionButton‘ for resolution displays the correct current setting.

Similarly, the ‘SetFullScreenIndicator‘ method ensures the window mode ‘OptionButton‘ reflects the current window mode. This method retrieves the current window mode and compares it against the values stored in the ‘Modes‘ array. If a match is found, the ‘index‘ is updated to the corresponding mode value, ensuring the ‘OptionButton‘ for window mode displays the correct current setting. These methods are crucial for maintaining consistency between the actual display settings and what is shown in the user interface, providing a seamless and intuitive experience for the user.

Developing the keybinding functionality in Atria was a crucial feature that allowed players to customize controls according to their preferences. In the context of a development with Godot, implementing keybinding offered several significant features but also presented various challenges.

Developing the keybinding functionality not only enhances user experience but also aligns with modern gaming standards where player customization is highly valued. Allowing players to tailor their controls means accommodating diverse play styles and preferences, making the game accessible to a broader audience. This feature, albeit challenging to implement, ensures that players can enjoy the game in a way that feels most natural to them, potentially increasing player retention and satisfaction.

Keybinding customization enables players to reconfigure controls to suit their preferences, enhancing the user experience by providing more

comfortable and intuitive gameplay. Moreover, this customization is especially crucial in competitive gaming scenarios where milliseconds count. Players often have muscle memory associated with certain keys and allowing them to set up their preferred controls can give them a significant edge. This not only levels the playing field but also promotes a more competitive and engaging environment. The keybinding system only supports the keyboard as input devices for the moment.

Additionally, keybinding configurations can be saved and loaded, allowing players to retain their preferences between gaming sessions. This functionality is essential for providing a consistent user experience. An intuitive user interface was developed for configuring controls, allowing players to easily modify settings through a dedicated options menu designed to be accessible and user-friendly.

First, the player sees a Label with the Action, and a clickable button with a text displaying the current bind associated with the action, if it exists. Additionally, to further streamline the process, I included a feature that displays the default bind for each action, providing a reference point for new players. This ensures that even if a player mistakenly unassigns a key or wishes to revert to the original settings, they can easily do so without needing to reset all controls. If it doesn't, the button shows "Unassigned". The player can then click on the button, and the text changes for "Press a New Key". The player then presses the key they want. If the key is already in used, the button shows "Already used" and the player can click on another key to rebind this action. If not, the action is set at a keymap level, which means that it is saved by Godot as a keymap for a certain action.

However, the development process encountered several challenges. Managing key conflicts, where two different actions are assigned to the same key, was a major difficulty. We also needed to add the current key to the Button, just in case the player wanted to know the current bind for a certain action without necessarily wanting to change it.

Testing and debugging the keybinding system required meticulous attention. Ensuring that all possible configurations worked as intended involved extensive testing. Designing a user interface that was both comprehensive and intuitive posed an additional challenge. It was essential to ensure that players could easily understand how to reconfigure their keys and that the interface was visually clear and coherent. That's why I created subcategories to guide the player in the keymapping, making the process of searching for a certain key to

remap much easier.

I quickly mentioned the fact that the settings are saved once changed. I oversaw this functionality too. The importance of saving settings cannot be overstated, as it ensures continuity for the player. Every time a player starts the game, they should feel that their preferences are respected, providing a sense of familiarity and comfort. This small yet significant feature plays a crucial role in maintaining user engagement and satisfaction. So, when the game loads, it applies the settings saved on a .Json file located on the user AppData folder. The game can read from the file and apply settings, as well as write the file once new settings are set. This was a very challenging functionality to add to our game, but the fact that the player is now able to load and save settings locally is crucial for the user experience.

To implement a parameter save system in my game, I created a static class called ‘SaveManager‘ that handles loading and saving of settings. This class uses a JSON file to store the settings, with the file path defined as a constant string ‘savefilepath‘. The path is set to ”user://settings.json”, which is a user-specific directory commonly used in Godot for storing user data. The ‘SaveManager‘ class contains two main methods: ‘LoadSettings‘ and ‘SaveSettings‘.

The ‘LoadSettings‘ method is responsible for reading the settings from the JSON file. It starts by globalizing the file path using ‘ProjectSettings.GlobalizePath(savefilepath)‘, which converts the relative path into an absolute path that the file system can use. It then checks if the file exists using ‘File.Exists(filepath)‘. If the file does not exist, the method returns a new instance of ‘ParamData‘ with default values. If the file does exist, it reads the content of the file into a string using ‘File.ReadAllText(filepath)‘. This string is then deserialized into a ‘ParamData‘ object using ‘JsonSerializer.Deserialize¡ParamData¿(content)‘, which converts the JSON string back into an object that can be used within the game.

The ‘SaveSettings‘ method is used to save the current settings to the JSON file. Like ‘LoadSettings‘, it starts by globalizing the file path with ‘ProjectSettings.GlobalizePath(savefilepath)‘. It then serializes the ‘ParamData‘ object passed to the method into a JSON string using ‘JsonSerializer.Serialize(data)‘. This JSON string is then written to the file at the specified path using ‘File.WriteAllText(filepath, content)‘. This method ensures that any changes to the settings are saved and can be reloaded the next time the game is started.

The ‘ParamData‘ class is a simple class that holds the settings to be

saved and loaded. It has two properties: ‘mute‘, a boolean indicating whether the sound is muted, and ‘soundLevel‘, a float representing the volume level of the sound. These properties have default values: ‘mute‘ is set to ‘false‘ and ‘soundLevel‘ is set to ‘1‘. By using this class, the ‘SaveManager‘ can easily serialize and deserialize the settings, ensuring that the state of these properties is maintained between game sessions.

Let’s go back to the main menu. Here, the player can press Play. When this is done, they are given the choice between an online game, which initiates random matchmaking with other players worldwide, or a custom game, which takes them to a menu where the user must enter an IP address and a port before being able to click on a Join button to launch custom games. Note that when the player decides to start an online game, they are connected to our game servers, and the connection between players is ensured by them, whereas if the player decides to play a custom game, one of the players will be the host of the game, and the others will connect to this game via an IP address and a port.

Before being able to click on Online or Custom, the player must refer a username that is at least 3 characters long and that has no space characters. This ensures no problems between the user and the game related to its username. This is done by constantly checking after each new input in the text box if the length of the username is at least 3 and if it doesn’t contain unauthorised characters.

When the player decides to play a custom game, they must enter an IP address and a port in two boxes. Fortunately, a default IP address and a default port are pre-filled, knowing that the default port will often be the one used, saving time and making it easier for the player.

To navigate through the game menus, players have the option to use their mouse or use their keyboard with the Escape Key. For players who prefer a more streamlined navigation method, keyboard shortcuts for the most frequently used actions were integrated into the interface. This feature caters to both experienced gamers who are accustomed to quick navigation and new players who may find it easier to use simple keyboard commands. The user can also return to the previous menu at any time by pressing the Exit button, which is present on all menu layers and can by either pressed or triggered by the Escape key.

All elements of the interface are 2D nodes of Godot, which have been arranged in the following manner in most cases: There is a Control node, containing a TextureButton which is itself containing a RichTextLabel. This implementation architecture was chosen because

it has many advantages for adapting the UI to various scenarios such as different screen resolutions, different screen ratios... Anchor Points have been placed around each node to resize them at will while maintaining the same ratio, allowing for a more natural and adaptive scene rendering. The adaptive scene rendering not only enhances the visual appeal but also ensures that the interface remains functional and aesthetically pleasing across various devices and screen sizes. This flexibility is crucial in today's gaming landscape where players may switch between different devices with varying resolutions. This architecture is also way more convenient in terms of style. The buttons are pretty, futuristic, and the text fits perfectly in it.

During the creation of these menu elements, various options emerged for their implementation. Initially, I considered creating a separate scene for each submenu of the interface, but I quickly realized that this would become cumbersome to maintain, and the confusion during development was too significant when dealing with all these scenes. This realization led to the adoption of a more modular approach, where components could be reused across different parts of the interface. This not only reduced redundancy but also streamlined the development process, making it easier to implement changes and updates across the UI. I then turned to a node-based system where I toggle visibility when I want to switch between submenus.

To switch between menus within the UI scene, C# functions had to be implemented to manage transitions between nodes. When buttons are pressed, a function associated with that button is called, triggering a program. The responsiveness of these functions was optimized to ensure that there is no noticeable delay between user input and the corresponding action. This is crucial for maintaining a smooth and intuitive user experience, as any lag or delay can significantly detract from the overall gameplay experience. For most cases, this is presented as follows: a variable is associated with a certain menu (represented by a node), another variable is associated with another menu represented by another node, and then I used the "variable.Visible" functions accompanied by Boolean values to configure the visibility of each of the menus.

In the body of the _Ready() method, which runs only once at the beginning, the command is given to display the "main" menu of the UI scene, to access the main menu and have access to all the functionalities.

The Atria user interface required the implementation of several

scenes to manage the display of elements on the screen. A first scene named "UI" gathers all the elements I have just talked about. Another one named "Lobby" displays a pre-match screen, where players can wait while viewing the mini-map, as well as being able to form their teams. I will address these points below.

The Lobby scene is displayed when the game starts, after the player has chosen between online or custom. On this interface, as mentioned earlier, in the middle is the minimap of the game about to be played (these changes constantly as the map is procedurally generated).

The menu also has two buttons "Add Team #1" and "Add Team #2". When these are pressed, new players in the form of Control nodes appear in the scene tree in a ScrollContainer node for each of the teams. This allows us to scroll through our teammates if their number exceeds the maximum that can be displayed by the size of the screen used.

To achieve this, I ensured that when the corresponding "Add Team" button for their team is pressed, the player's card is loaded and placed into the ScrollContainer. The player's card is created in a fourth scene, named Lobby_Player_Template, which allowed us to aesthetically design a card that will be called by the function and placed into the ScrollContainer. Then, depending on the player's team and thus the button they pressed, they will see a new player added to their team.

Finally, regarding the HUD, which is all the visual elements available to the player to communicate information about their equipment, character, or the ongoing game, I have finished developing both an interactive health bar and an energy bar showing the player's remaining health points as well as its energy. The bar empties based on the amount of damage the player takes, or the actions they make. If the health bar drops below 50%, the colour changes to yellow, and below 15%, it becomes red. If the energy drops below a certain lever, it starts to blink.

To implement these bars, I used a vector to assign new X and Y coordinates values to the bars. Additionally, the visual indicators on these bars were designed to be easily interpretable immediately, using not just colors but also icons and animations. This multi-faceted approach ensures that players can quickly and accurately assess their status, even in the heat of battle. I ensured that the X position, on the horizontal plane, is shifted to the left by N pixels when the player takes N damage for the health bar and the energy bar starts to blink in a same way, but vertically.

The colours of the health bar were assigned to correspond to states of good, poor, and critical health, which are represented by indicative colors. To do this, once again, I use the X position of my health bar, and I check each frame if it falls within certain values.

Then, I assign a certain colour to this ColorRect based on its position. This is then moved Nelson's gameplay scene, allowing it to be used in a scene where it will be useful.

Moreover, on the very first menu, there are two other elements of the user interface. The first one is the logo of our game, Atria, placed in a white ColorRect, and the second one is a dynamic changelog.

The second one is a changelog box. The changelog uses the GitHub API to get the Name of the latest release as well as the description of the release. This can be interpreted as a patch note space, where the user can see the latest updates and changes of the game.

2.2 Assets

As mentioned earlier, all elements available to the player, whether HUD elements or interface elements such as menus, must be functional and intuitive. But that's not all. The user interface doesn't stop at the functional part of the elements displayed on the screen; an aesthetic part also comes into play. So now I will talk about assets, which are the elements used to embellish the textures, materials, and shapes of the objects and interface elements of the game.

The search for assets for our game was based on the quest for an immersive experience. As you may know, the theme of our game revolves around the galactic and space universe, as well as everything that includes the futuristic domain. Our research efforts therefore turned towards assets bringing together these different elements, while seeking for sobriety in them.

The goal through the search for these assets is to create harmony among all elements of the game, whether it's the menus, interface elements, weapon designs, map textures, environmental elements, or decorative objects. Immersion lies at the heart of our project, and our ambition is to craft an environment that is coordinated and in harmony with every other aspect of the game.

That's why the background of the menus is a dark sky with stars, and the buttons have been stylized to give them a sleek look. I also designed the characters of the game to implement them on the website, providing a glimpse of the game right from the download page.

I chose a futuristic-like button that is used everywhere in the menus.

The button is dynamic, I edited it to make three variants out of it. The first one is the basic button, when the cursor is not on the button. The second one is when the cursor is on the button, but the user is not clicking on it. The button changes its appearance, becomes smaller and I added style elements around it. If the player clicks on the button, a variant of the second style is displayed, informing the player that they are interacting with the button.

3 Nelson Rudent

3.1 Gameplay

Once joining a game from a lobby of the Main Menu that Guillaume created, a game must be started. Yet before it runs on a server, it should have a playable structure to operate on the player's machine. This is where the enjoyability and its overall ability to perform on sales is determined. Indeed, the core of any video game lies in its gameplay mechanics, which defines how players interact with the environment given and the experience they get from it.

As Atria is a first-person shooter (FPS), the very base of the game begins by a character who is able to perform essentials movements interactions with an environment, such as the map or other entities. Hence, I created a class representing a player which can be used for both the character that we, as a user, would be playing, and the ones that are manipulated by other users. It first includes the movement mechanics by simply wandering around and jumping while always being able to hit and stop at possible collisions such as the ground.

Even if those are sufficient for a classic FPS, the futuristic theme of the project invited me to expand the player's motion by adding extensions to the route movements. Thus was built variations of the velocity for the motion and a double jump enlarging the possibilities offered by the jump action. The motion now embraces a walking velocity as well as a running, sprinting and crouching one. The latter comes with another mechanic that will be explained farther in the gameplay presentation.

Each type of velocity, stored within a struct class, can be accessed and used for the character's motion with a unique keyboard key, allowing them to switch between them. However, some are dependent of one another. Indeed, you cannot run, sprint or go forward while crouching if you do not have the walk key pressed. Yet, you can use the sprint mode while running, which will override your running speed. Similarly, the crouching speed is dominant on all other modes. The motion is done by deconstructing the velocity vectors and applying a MoveAndSlide() physics method offered by Godot's Native System. It is written in such a way as to provide the best realistic movement experience and is one of my finest achievements. It even comes with an acceleration integrated, which takes into account air resistance, making a smooth in and out movement whenever a change happens in the player's velocity.

As for the double jump, it is independent of a ground collision.

It means a pre-earned velocity on the ground axis will be kept when double jumping which is not the case on collision of the ground. In truth, I was not yet able to implement a sliding mechanism, with the player's intent or without, which makes it impossible for it to work on both running before crouching and jumping with forward velocity. Talking about crouching, it is a powerful mechanic that I looked into. I developed it by shrinking the player but at an offset origin point in order for the base of the character to always be in touch with the ground or other corresponding position. To complete the crouching process, often known as shifting in the gamer world, I added simple animations that instantly enlighten the experience of the mechanic from my point of view. I learned a lesson from it, understanding that the quality and utility of a mechanic may be hidden by its poor display graphics. The following nodes attached to the character as body parts are also affected by position of the crouch mode but not all by size.

To not forget as one of the essential mechanics for Atria to be an FPS, I needed to include the head, or as we call it in its formal term, the 3D Camera. I connected the mouse to its rotation but clamped it on a range for the vertical axis as a normal head cannot loop around the neck. It is also positioned perfectly to not visually view through the player's own body, yet it being still visible for example when looking down at their feet. Even if I succeeded that task, it caused me great troubles to pull off.

To delve into more advanced mechanics of the game, it was now time to produce the "shooter" part of Atria. Before having any weapons, a hand to hold it was needed and I linked it to the head of the character for it to move accordingly when crouching and looking around. As we did not prioritize assets yet, there is no 3D model for any kind of weapon even if the concept descriptions of some exist, I decided to test it with an asset of a AK-47 model given by Guillaume. As for the processing part of the weapon, it needs its primarily objective: shooting bullets. As I am also in charge of designing the concept for each and every part of the game, I knew before working on it that most of the concepts for the weapons do not use bullets.

Thus, I skipped the process of crafting a visual bullet shot by the weapon. Instead, I used a ray-cast as the aim to catch the collider, whether it is a wall or an enemy. However, there is no camera effect yet such as a shaking effect, nor is there a random component in the aim which greatly decrease the shooting experience and is one of the key elements disappointing me for now. In order to correctly test it,

I decided to implement an enemy dummy with health and a death detection. It could be used in the future for non-playable characters (NPCs). Everything was going well but I could not guess where was I hitting with my gun. Although I said the concepts of the weapons in Atria is nowhere near similar to today's classic FPS, I am only in the testing phase of them. Hence, I concluded that a bullet decay would help visualize the aim. As of today, the bullet decay does not work on the ground only due to vertical axis problems.

To enhance the experience of the weapon and to promote the skills needed to use it, I implemented and designed simple animations of firing and aiming. The prior one controls the firing speed of the weapon, only shooting in the intervals of the animation. However, they both conflict each other but they will be adapted once the correct assets are imported as the animation are dependent of the 3D Model's shape. Some simple sounds were added to improve the player-environment interactions which includes basic fire noises but restricted to a family-friendly content.

3.2 Multiplayer

The local structure of the gameplay is enough for a player to be able to run Atria on their own device and have the features working correctly. Yet, our project is a multiplayer game, not a singleplayer one. Hence, before even hosting it on a server, it requires all of the features to be compatible with the Multiplayer branch. It inevitably builds itself on the Gameplay branch. A revisiting of every interactions feature, which are mostly not involving graphics, was required. To link the player's integrity to a server, I derived the character's parent to a new custom class wrote by the Multiplayer's Responsible, Lilian. This allowed me to handle the problem caused by the head rotation. Indeed, while it worked in singleplayer, it does not when two instances of the same class, the player, are invoked. This is because Godot sends an input event to all of the instances, even if the motion of the mouse of one player only comes from a single entity. In-game, the weapon of the enemy took the same rotation as the user. Using the method from the server class enables me to separate the two instances.

The game experience being somewhat sufficient, we decided to move onto the next step of the Multiplayer renovation. This included killing the enemy player, or dying ourselves. Before any detection of these two, I implemented a encyclopedia for the possible types of death cause. This will be used in the future to send to the server for it to

calculate and generate all sorts of statistics and information needed. We do not plan to host those processes locally to reduce and minimize the possibilities of hacks and bugs, even if we know that, as a company with a low budget, we will not be able to defend ourselves efficiently against such encounters of cheaters.

As the death was classified, I tried detecting the death event of a player. However, after analyzing the game's logs, this task was not successful as the result turned out to be in reverse: whenever you killed an enemy, the game considered that you mysteriously died. This demoralized me due to it being an important part of the gameplay. It shall be fixed in the close future by syncing again to the server side. The Multiplayer branch was not as prioritized as the Gameplay branch in those last months as we needed a strong and sufficient base done before building the multiplayer side. However, this order of importance will change soon as I already constructed an adequate structure. Henceforth, the new Gameplay features will be built with a pre-installed server connection.

3.3 Game Design

Regarding those so called Gameplay features and mechanics, their implementation is not the first action taken. Indeed, a feature requires a concept in the first place, a possible design and some thoughts into the ways of creating it. While most of the current ones did not go through this process, it was only due to them being the foundation of a first-person shooter game. An example of a feature that did had to be conceptualized was the movement mechanics which does not necessarily copy the common form of other games. Hence why designing represents a considerable role in the creation of Atria, or any other fancy video game. It is commonly referred to as the branch of planning because everything done within it represents what other branches have to implement in the future. Thus, everything described in the following also constitutes what needs to be done for the next time.

While we have only designed for now the game without a proper goal except the fundamental one, killing the enemy, there is an actual story behind it, which was summarized at the defense of January, that supports a more complex purpose. I am addressing the matter of game modes. There are three of them which are unique to Atria. The first one is the minimum we expect to finalize for the end of the year: Dominion. I designed this game mode to be divided in 15 rounds.

This decision was made upon analysis of multiple famous FPS in the video game industry where a round-based match generally gave more satisfaction and dopamine than other types of matches.

This game mode, just like the two others, represents a crucial part of the war described in the Story Game. Dominion, for which the name is derived from dominance, is the title given to attacks by one of the two clans on a specific critical check point. The attacking side engages a battle with stealth by infiltrating the enemy base and taking control of it. In-game, this part of the story is an electrogenerating bomb which wipes out the enemy technology. This device must be installed and activated at a target to end the round which gives a somewhat similar experience of the classic defuse game mode present in multiple famous FPS such as CS:GO.

However, it is portrayed in a stealth situation, giving players a much different approach. The stealth mechanism would be the last step that we will implement to complete this game mode. A round ends once a bomb detonates, is defused or the infiltrating team is killed. The game ends when a team reaches the threshold number of rounds won, which would be the main implementation priority after we create a normal round with only a team wipe out as a possible win. As most games which includes a similar mode, we are too going to implement a "half-time" where teams switch positions. However, it will happens much differently in Atria. At the end of every round, if the attacking side succeeds in taking over the enemy camp, they switch sides. Although this does not necessarily promote a fair gameplay between the players, we believe it will bring a more living atmosphere for our players.

Of course, the players will not be given one weapon to use all the time. Weapons are an important part of the gameplay and the experience users get from the game. After completing the scoring mechanism of the rounds, we will be able to focus independently on the armory. This includes finding assets, creating some by modeling, texturizing and animating, and implementing their special features. Indeed, I have also written concepts of weapons to initiate an entire new armory that new players will discover. They will possess different required mechanics to master and will allow more strategy possibilities.

Here, I am talking about different types of them as we do not have a precise number of numbers of weapons, nor names, nor profiles for them. To mark the futuristic theme, I conceptualized plasma weapons first. They shoot a powerful laser beam but are unwieldy. Requiring an extended duration to reload, they will be balanced also by the lim-

ited ammunitions given, if those can still be called the same in the future. For a dramatic effect, there will also be radiation weapons classed in two subtypes. The first one is the most similar type of weapons to what we possess today: it shoots radioactive bullets, less impact resistance but a continuous damage inflicted after hit. The second however, completely differs from the bullet-based concept: it produces a strong radiation field affected short-range targets. Speaking of fields, electromagnetic weapons are also planned to be added. Such guns will either be a powerful shock close to shotguns or an advanced middle-range crossbow whose projectiles tend to link with each other producing an electric shock for any entity in-between them. Finally, there will be close-range weapons, or melee weapons, such including but not limited to laser wands, an amplificated fist and perforating retractable blades.

To complete with the design, the different characters available for each game will possess special technologies, not to confuse with abilities. This part of the branch will be the least prioritized component of the game as we will focus on building a functional project before implementing these advanced features. However, I still already conceptualized their general idea with some examples. A character class represents the training that the soldier had as well as the technologies they are given in battles. With this structure, each character can be chosen but also customized in terms of aesthetic by the user. This approach provides a bridge between ability-based characters from games such as Valorant and a customized-based class as seen in Call of Duty. With these two process united, we bring out the benefits from each sides with the abilities promoting strategy while allowing customization to please the users.

The customization section of characters are a mix of cosmetics and profiles. A player can entirely change the outline aspect of each character they have unlocked, given it a unique look while still having the same shape of their class. Furthermore, as the game host their battles in a futuristic theme, players will be given advanced recognition systems procuring them the necessary information to figure out the character class of each enemy, eventually telling them what technology they possess.

On the other hand, the technologies given to each character, which are called modules, will be in numbers of three per class. Each of these require a specific and unique amount of energy to be used. Players will possess an energy bar on their screen which can be filled with various

actions depending on the game mode, such as killing. The energy can be used on multiple different modules or on a single module multiple times. It is all up to the player. These modules can only be upgraded aesthetically by cosmetics: it would be unfair otherwise. On top of three unique modules per class, characters will be equipped with an ultimate which is a module more advanced than the others. It can only be activated when the player's energy tank is full and that they are in possession of an Atria. For reminder, an Atria is a device implemented in every human's heart of the population which allows them to survive in the Solar system. It is the mode advanced device of humanity and is coveted for its phenomenal energy production.

4 Lilian Labat

4.1 Procedural Generation

To be able to play this targeted game mode the game needs a map. The map that as been selected to be developed for the first stage of the game is a map based on a grid, each tile on the grid represents a corridor that can be selected from a fixed set of tiles. This type of map as been selected because it is the easiest and fastest system to implement, and it is very reliable and customizable. It is the best system to obtain interesting and enjoyable maps in a realistic time scale of development for our team. It is although compatible with multiple game modes whether it is for a symmetric or asymmetric game mode the generation can be adapted easily.

To support this system of map generation we use an algorithm called "wave collapse function", this algorithm can fill a tile map (a grid of tiles) with coherent tiles depending on what is already on the tile map. To do this it uses a set of rules to know what tile can be placed. In Atria the tiles are corridors connected to each other, so the set of rules corresponds to, for each tile, on which side of the tile a connection to another corridor is needed. For some tiles there are some additional rules corresponding to the rules to change level.

By default, the "wave collapse function" algorithm does not handle 3D grid, but in Atria the map is a 3D grid because there are multiple levels of corridors. To do that we used the algorithm for each level independently and when a junction is needed between levels the junction tile is added on all affected levels. The system has been built to be highly modulable, therefor if new tiles are created, we can add them to this system easily, if we want to adjust the different parameters of the algorithm, we can also do it.

One of the biggest advantages of this system is that we can place tiles before calling the algorithm and the algorithm will build around it with no problems. It allows us to place tiles on the map using other method. For example, the game mode Dominion need to have rooms for the player to place a bomb to win the match, those bomb's rooms can be placed by a script before calling the algorithm and the algorithm will be able to adapt to the placement of those rooms. This is the next step for the procedural generation, we need to find and implement a system to place those bomb's rooms on the map. We also need to implement a system to place the spawn of the teams. The spawn of the team in the game mode Dominion will be placed on the edge just outside the tile map and the "wave collapse function"

algorithm needs to know where the spawns are to be able to connect the spawns correctly.

Lastly the procedural generation also need a system to be sure that every point of interest, spawns, bomb's rooms... are all connected to each other and that all teams can access every point of interest. We are planning to use the A* algorithm from the IA part of the game to check if every room are connected and if some are not then the system generate a new map until everything is connected.

This method is potentially very slow because it may need many generation of the map to finish but the current map generation take a few seconds at most to generate a map of decent size, further more the generation of the map is done only on the server and it is done asynchronously that means that player can join the server while it is generating a map so the server can generate the map during a time that is already a waiting time because the server needs to wait for all the clients to connect before starting the game. This system has been chosen mainly because it maintains the random and interesting patterns generated by the "wave collapse function" algorithm. Some other alternatives are still being considered but they are a lot more complex to implement and may result in loosing the randomness of the generation, so we will need to experiment with all those systems to choose the best one for Atria.

To conclude on the procedural generation, its main mechanics are implemented, and we can generate interesting maps for the player. In addition, the implementation of the bomb's rooms and the spawn will be easy to do because of the chosen generation system. The system that makes sure that the map is correct for the game mode need reflexion and work to be implemented. However, this state of advancement on the procedural generation was expected and we are on schedule regarding what was announced on the book of specifications.

4.2 Multiplayer

For the multiplayer part, the implementation of the multiplayer is highly dependent on all the other parts of the game. The goal the multiplayer is to implement all the other functionality of the game and make them synchronised between all the players. It is why the focus of the multiplayer part has been to coordinate all the different part of the project to replicate them online and make everything works with the multiplayer system.

The multiplayer is especially link with the match management

which is why the two parts work together to allows the game to work and create the base of the match management. The current state of the match management is that the first part is the lobby will every player is connecting, and the server is loading the map, then when all the players have connected, the players are spawned on the map after a little delay to the players to be warned. Then the match begin after a certain amount of time.

It started with the UI, for the multiplayer to initialise it needs multiple information, for example if we want to start a server or a client, and what ip address and port to use. This is managed mainly by the UI so we created a way to link the UI want the multiplayer using events, for example when a player clicks on the play button the UI call the multiplayer and tells it to lunch a client, then the multiplayer can connect the client and start working.

The first things the multiplayer does went connecting to a server is to transmit the player's data, for now it is just the username, but we can transmit what we want. In the lobby the multiplayer communicates between the server and clients to synchronise the connected players data to display it on the UI. It although transmit the map from the server to the clients when the map is finished generating.

When the server is full it will lunch the game, the multiplayer spawns all the players on the server and on all clients. From there it oversees synchronising the movements and rotation of the player as well as all the actions like jumping, firing...

To be able to synchronise the movements of the players, the multiplayer needs to apply many modifications to the player movement managed by the Gameplay.

To simplify this process, we decided to split the multiplayer management and the local movement into two objects, all the local movement managed by the Gameplay part is done in the player class and all the multiplayer management happens in the local entity class, and the local entity class calls the player class when the local player needs to do movement or actions.

The local entity class is call so because to minimise local lag, all the movement are done in local by the player then the local client sends the data to the server which sends this information back to all other players, which means that all movement computations are done in local, whereas a server entity will be moved by the server then synchronised on all clients.

But this method has a big flaw which is vulnerability to cheats,

the client's information can never be thrusted that is why we will need to implement an error correction algorithm that will check if the movement done by the player is possible, if not the server needs to put the payer back on possible path. In addition, some anti-cheat system needs to be created for the firing system.

In addition to the implementation of the different features of the game the multiplayer part implemented a multiple way to manage and deploy servers. To start with, it is now possible to start a server using command lines and the headless mode of Godot that deactivate the rendering part of the game. To do this the multiplayer allows the user to start the game with custom command line arguments such as: "–server" which will automatically start a server for the game, the parameters of the server can be changed using command line arguments such as: "—nbPlayers x" which will modify the number of players in the server to the x value. The list of all custom command line arguments is available on the README file in the source code or in the release archives.

Another way to deploy a server for Artia is to use a docker image. We added a dockerfile in the source code to allow anyone to easily create a docker image containing the game, the default dockerfile will automatically lunch a server with parameters when a container is run but the dockerfile can be modified to put other parameters or to copy parameter file.

The dockerfile simply take an image of Ubuntu with godot installed and build the project using godot, then remove godot and the source code and run the game. This system allows anyone to build a docker image containing its personal server configured correctly and then the person can send this image to any server or cloud that is able to run docker containers to deploy its server anywhere.

Finally, to allow easy access to the server's parameters event when the game is not running the user can use the "–saveParams" to create a json file containing all the server's parameters. The user can also manually create this file in the game's folder. The server will automatically use the parameters in this file instead of the default parameters. When command line parameters are passed to the game the command line parameters override the file's parameters.

To sum up, all the current functionalities of the game are implemented for the multiplayer and the multiplayer is just missing some anti-cheat functionalities, but some additional features have been added. Therefor the multiplayer part is following along the other func-

tionality of the game with no problems and thus we are on schedule for the multiplayer part. While waiting for other functionalities to be created the multiplayer will focus on implementing the anti-cheat functions.

5 Luan Ray-Chagas

5.1 Sound

Atria is set in a dark and macabre futuristic world. To ensure optimal immersion of players in this universe, several aspects are taken into account, including sounds and atmosphere. They play a crucial role in creating a deep immersion for players. Two distinct soundscapes have been created to enhance the game's atmosphere.

The game lobby is the players' first experience of Atria's universe, and it's essential to capture their attention and immediately immerse them in the game's dark and tense atmosphere. For this purpose, a crucial decision was made regarding the choice of background music. A trap melody was specifically selected for its distinct sonic characteristics that align perfectly with the desired atmosphere. Trap is a modern musical genre emerging from the hip-hop scene. It is recognized for its deep bass, syncopated rhythms, and often sinister or unsettling melodies. These sonic elements are perfectly suited to Atria's universe, where a dark and oppressive atmosphere reigns supreme. The catchy rhythms of trap immediately grab players' attention, plunging them into a state of tension and readiness for imminent action.

By choosing a trap melody for the lobby, the goal is to immerse players in the world of Atria from the moment they connect to the game. The music acts as an emotional bridge, evoking feelings of urgency, danger, and excitement. The impactful sounds and deep bass create an oppressive atmosphere that puts players in the right mindset to face upcoming challenges. In addition to creating an immersive atmosphere, the trap music reinforces Atria's distinctive visual and sonic identity. It helps define the tone and style of the game, allowing players to feel immediately immersed in its unique universe. This consistency between music, visuals, and gameplay creates a holistic experience that captivates and engages players from the very beginning.

The loading screen in Atria represents a crucial moment where players eagerly await the start of the game. To maximize engagement and mentally prepare players for the intensity of the upcoming gameplay, a strategic decision was made to use a rock melody as accompanying music. Rock has long been associated with action scenes in media, whether in movies, television series, or video games. Its catchy rhythms, energetic electric guitars, and impactful lyrics evoke a sense of excitement and adrenaline. By choosing a rock melody for the loading screen, Atria capitalizes on this association to prepare players for

the frenetic action ahead.

The rock music played during the loading screen acts as a mental stimulant for players. It's contagious energy and dynamic rhythms encourage players to mentally prepare for the intensity of the upcoming gameplay. Roaring electric guitars and powerful percussion create an atmosphere of action and suspense, further stimulating players' excitement before the start of the game. By using a rock melody during the loading screen, Atria reinforces the game's atmosphere of action and suspense. This energetic and dynamic music creates a progressively escalating buildup, preparing players to dive into the action from the very beginning of the game. It also helps maintain players' engagement during waiting phases, encouraging them to remain focused and ready to face the challenges ahead.

The creation of music for Atria was carried out using online studios, offering a practical and cost-effective approach to achieving quality results while adhering to the project's budget constraints. The rock melody played during the loading screen was produced on BandLab, a free online studio. BandLab provides a range of powerful tools for music creation, including virtual instruments, pre-recorded loops, and advanced editing features.

By using BandLab, the Atria development team was able to create a dynamic and engaging rock melody without the need to invest in expensive software. In contrast, the trap melody used in Atria's lobby was produced on FL Studio, a professional paid music production studio. FL Studio is widely recognized in the industry for its versatility, user-friendliness, and advanced music creation capabilities. Although FL Studio requires an initial investment, it offers more advanced features and greater flexibility compared to free solutions like BandLab.

By opting for online studios, the Atria development team was able to balance the quality of the music with the project's budget constraints. BandLab allowed for the creation of a high-quality rock melody without additional costs, while FL Studio was used to refine the trap melody in the lobby, providing an additional level of sophistication to match the desired atmosphere.

The use of online studios also allowed the Atria team to maximize available resources. By leveraging the tools and features offered by these platforms, they were able to produce original and captivating music that perfectly matched the atmosphere and style of the game, all while staying within the allocated budget.

5.2 Networking

Having explored the important role of sound in the game, we know the immersion aspect of the game is crucial. On the other hand, it would be a shame if an unbalanced match of the players ruined it. This is handled by the Networking branch which provides a matchmaking process to resolve this issue.

Matchmaking in Atria is managed by a C# console application that utilizes the TCP protocol to communicate with game servers. To provide an overview, when a new server is created, the matchmaker records information such as the maximum number of allowed players. It updates server information in real-time, including the number of connected players.

When a server is no longer active, the matchmaker removes it from the list of available servers. On the other hand, when a player wishes to join a game, the matchmaker returns the ID of a server where the maximum number of players has not been reached, ensuring a balanced distribution of players across different games.

Networking in Atria is essential to ensure a smooth and immersive multiplayer experience. It encompasses all the technologies and protocols necessary to enable communication between players, synchronize real-time data, and manage connections. In Atria, networking ensures that players can interact in real-time within the game's virtual environment. This includes transmitting character positions, actions taken, and other relevant data to ensure a coherent and engaging gameplay experience.

To understand the concepts of matchmaking and networking, we watched YouTube videos to get a better idea. These videos showed us how matchmaking works and how players communicate in games through networks. But even with all this, it was tough. The details of matchmaking and networking were tricky. We had to experiment a lot, trying different things to see what worked. We had to figure out things like how to make matches fair and how to fix it when players couldn't connect to each other. Understanding how everything fits together was the hardest part. It took time and effort to get it just right.

Conclusion

Throughout this report, we have examined the contributions of each team member, highlighting their invaluable role in advancing various aspects of the project. Every problem faced, every solution granted, every success was greeted with pride of our group project as we all share a common satisfaction in how far we have been able to develop such a game.

Looking back on our achievements, we take pride in the milestones we have reached and the challenges we have overcome. We have laid a strong foundation for Atria, one that promises an engaging and unforgettable gaming experience for future players.

Moving forward, it is essential to prioritize certain areas to ensure the continued success and growth of Atria, especially since some were given less attention so far. Enhancements to gameplay mechanics, refinement of multiplayer features, and the expansion of the atmosphere including assets and user interface elements should remain at the forefront of our development efforts. Additionally, ongoing optimization and bug fixes for networking and procedural generation will be essential to delivering a polished and seamless product for the end of the year.

Thank you to our team members for their dedication and hard work, and to you for reading this report. With determination and passion as our guiding forces, we put high expectations on the future of Atria, which is a project that allowed us so far to outgrow ourselves and better learn how to work in groups.

Annexe

- AI-Generated menu background.
- Royalty free and customized buttons.
- Gun 3D Model : AK-47.
- Map Models : Free SpaceStation Assets.
- Background Music by Luan Ray-Chagas.
- Free bullet holes asset.
- Free AK-47 shot sound effect.